Valovna enačba

Izabela Ostanek

Junij 2026

1 Uvod

V okviru projekta pri predmetu Matematično modeliranje sem se ukvarjala z numeričnim reševanjem in vizualizacijo valovne enačbe v dveh dimenzijah. Cilj projekta je bil ustvariti interaktivno simulacijo, ki v realnem času prikazuje širjenje valov, npr. na vodni gladini ali vpeti membrani.

2 Valovna enačba

Osnova modela je parcialna diferencialna enačba drugega reda:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \Delta u$$

kjer spremenljivke predstavljajo:

- $u(t, \mathbf{x})$ funkcija časa t in prostorskih koordinat $\mathbf{x}$
- c – hitrost valovanja
- Δ – Laplaceov operator, ki opisuje, kako okolica vpliva na višino posamezne točke

Ker se ukvarjamo z dvema dimenzijama, lahko Laplaceov operator v kartezičnih koordinatah zapišemo tako:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right).$$

3 Diskretizacija prostora

Da bi problem rešili z računalnikom, moramo prostor diskretizirati. Celotno območje razdelimo na mrežo točk velikosti $N \times N$, kjer za vsako točko spremljamo njeno trenutno višino u in hitrost v.

3.1 Preoblikovanje enačbe v sistem prvega reda

Ker standardni algoritmi, kot je Runge-Kutta, znajo reševati le enačbe prvega reda, našo originalno enačbo drugega reda preoblikujemo v sistem dveh enačb prvega reda z uvedbo nove spremenljivke hitrostno polje:

$$v_{i,j} = \frac{\partial u}{\partial t}(t, x_i, y_j).$$

To nam omogoča, da ločeno izračunamo premik točke in spremembo njene hitrosti glede na sosede.

$$\begin{aligned} \dot{u}_{i,j} &= v_{i,j} \\ \dot{v}_{i,j} &= \frac{c^2}{h^2}\left(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}\right) \end{aligned}$$

4 Numerično reševanje

Sistem bi sicer lahko reševali z enostavno Eulerjevo metodo, vendar sem se zaradi večje natančnosti odločila za metodo RK4.

Metoda RK4 v enem časovnem koraku izračuna štiri različne koeficiente in uporabi njihovo uteženo povprečje. To zagotavlja boljšo numerično stabilnost, kar pomeni, da simulacija ostane natančna tudi pri daljšem izvajanju in hitrejših spremembah.

Definiramo matriki U in V. Matrika U vsebuje odmike $u_{i,j}$, matrika V pa hitrosti $v_{i,j}$. Skupaj opisujeta celotno stanje valovanja v vsakem trenutku.

Za vsak časovni korak Δt izračunamo štiri pomožne koeficiente za obe matriki U in V:

$$\mathrm{K}_1^U = V, \qquad K_1^V = f(U, V)$$

$$\mathrm{K}_2^U = V + \tfrac{\Delta t}{2} K_1^V, \qquad K_2^V = f\big(U + \tfrac{\Delta t}{2} K_1^U,\ V + \tfrac{\Delta t}{2} K_1^V\big)$$

$$\mathrm{K}_3^U = V + \tfrac{\Delta t}{2} K_2^V, \qquad K_3^V = f\big(U + \tfrac{\Delta t}{2} K_2^U,\ V + \tfrac{\Delta t}{2} K_2^V\big)$$

$$\mathrm{K}_4^U = V + \Delta t\, K_3^V, \qquad K_4^V = f\big(U + \Delta t\, K_3^U,\ V + \Delta t\, K_3^V\big)$$

kjer funkcija f predstavlja izračun pospeška.

Končni posodobitvi matrik sta:

$$U_{n+1} = U_n + \frac{\Delta t}{6}\left(K_1^U + 2K_2^U + 2K_3^U + K_4^U\right)$$
$$V_{n+1} = V_n + \frac{\Delta t}{6}\left(K_1^V + 2K_2^V + 2K_3^V + K_4^V\right)$$

4.1 Dušenje

V model sem vključila tudi dušenje, ki je sorazmeren s trenutno hitrostjo točke:

$$\dot{u}_{i,j} = v_{i,j}$$
$$\dot{v}_{i,j} = \frac{c^2}{h^2}(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}) - kv_{i,j}$$

kjer k predstavlja koeficient dušenja, kar omogoča, da se valovanje sčasoma naravno umiri.

5 Implementacija

Ustvarila sem interaktivni program v Pythonu, ki prikazuje delovanje valovne enačbe na $N \times N$ polju. Pri razvoju sem najprej uporabila knjižnico `matplotlib` za prikaz simulacije in `PyTorch` za izvajanje matričnih operacij. Pri večjem številu točk je takšen način prikaza postal manj primeren za interaktivno simulacijo, zato sem grafični del programa kasneje nadgradila.

Trenutna različica uporablja `PyQt6` za izdelavo grafičnega vmesnika, `PyQtGraph` za 2D prikaz in `PyQtGraph OpenGL` za 3D prikaz valovne površine. Program omogoča sprotno prikazovanje spreminjanja površine ter interaktivno dodajanje valov in ovir.

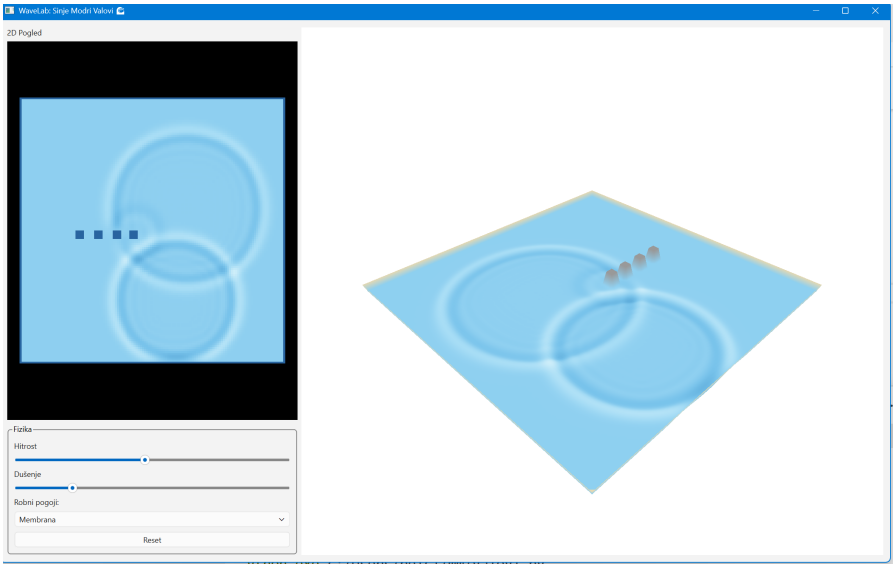
5.1 Optimizacija

Pri računanju valovne enačbe je pomembno, da se operacije izvajajo čim bolj učinkovito, saj je treba v vsakem časovnem koraku izračunati vrednost za vsako točko na $N \times N$ mreži. Namesto da bi računala vpliv posameznih sosednjih točk z več ločenimi operacijami, sem uporabila konvolucijo z Laplaceovim operatorjem.

Laplaceov operator predstavim z jedrom

$$L = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

Knjižnica `PyTorch` vsebuje funkcijo `conv2d`, ki to jedro aplicira na celotno matriko U. Tako lahko naenkrat izračunamo prispevek vseh sosednjih točk in dobimo približek prostorske ukrivljenosti valovne površine.



Slika 1: Prikaz vmesnika interaktivne simulacije

Takšen pristop je učinkovitejši in omogoča, da se računanje izvaja z matričnimi operacijami namesto z večkratnim ročnim obravnavanjem posameznih točk. Trenutna različica programa fizikalni del simulacije izvaja na procesorju, medtem ko se za 3D prikaz uporablja OpenGL. V prihodnosti bi bilo mogoče izračune s knjižnico `PyTorch` prenesti tudi na grafično kartico, na primer z uporabo CUDA, če bi bila na voljo podprta grafična kartica.

5.2 Robni pogoji in ovire

- Fiksni robovi: simulirajo vpeto membrano, kjer sta odmik u in hitrost v na robu ves čas enaka 0.
- Prosti robovi: Omogočajo prost odboj valovanja na robovih mreže (npr. vodna gladina v bazenu).

  Na robovih in v ogliščih zaradi manjkajočih sosedov Laplaceov operator zavzame prilagojene oblike in sicer:

  za oglišče $u_{1,1}$ velja:

$$\begin{aligned} \dot{u}_{1,1} &= v_{1,1} \\ \dot{v}_{1,1} &= \frac{c^2}{h^2}\left(u_{2,1} + u_{1,2} - 2u_{1,1}\right) \end{aligned}$$

  za robne točke $u_{1,j}$, kjer velja $2 \leq j \leq n-1$, pa:

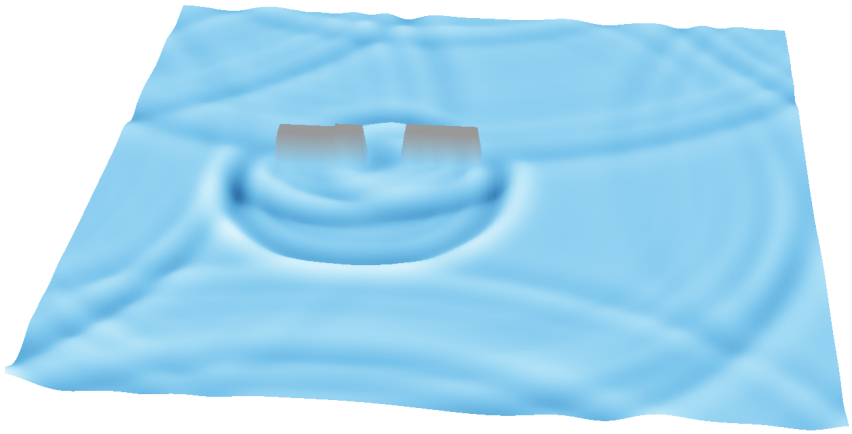
$$\begin{aligned} \dot{u}_{1,j} &= v_{1,j} \\ \dot{v}_{1,j} &= \frac{c^2}{h^2}\left(u_{2,j} + u_{1,j+1} + u_{1,j-1} - 3u_{1,j}\right) \end{aligned}$$

V trenutni implementaciji se tem prilagoditvam izognemo tako, da vrednosti na robu matrike preprosto prekopiramo navzven, s čimer ustvariva navidezne sosede. To nam omogoča, da celotno mrežo hkrati izračunamo z isto konvolucijsko matriko, kar poenostavi kodo in pohitri izvajanje.

- Ovire: Dodala sem tudi možnost interaktivnega postavljanja ovir. Za vsako točko (i, j), ki predstavlja oviro, nastavimo:

$$u_{i,j} = 0, \qquad v_{i,j} = 0$$

To omogoča simulacijo uklona in interference.



Slika 2: Prikaz simulacije interference

6 Povratne informacije s predstavitve

Po predstavitvi sem prejela predlog, da bi povečala reže pri demonstraciji, da se bolje vidi interferenčne vzorce, kar sem potem tudi upoštevala. Eden izmed sošolcev pa je pohvalil vizualni izgled simulacije.

7 Viri in uporaba AI

Osnovne usmeritve za projekt sem pridobila na vajah pri predmetu Matematično modeliranje ter na uvodnem sestanku z nosilcem projekta.

Pri implementaciji sem si pomagala z dokumentacijo knjižnic `PyTorch`, `PyQt6` in `PyQtGraph` ter z orodjem umetne inteligence Gemini. Dokumentacijo sem uporabljala predvsem pri uporabi knjižnic in izdelavi grafičnega vmesnika, Gemini pa predvsem kot pomoč pri optimizaciji implementacije in odpravljanju težav pri razvoju programa.